Operating Systems Lab — Viva Preparation

Assignments 1–8 · 2026-27 · Key theory, essential code snippets, and answers to each assignment's FAQs/viva questions

1. Shell Programming — Student Database
2. Process Control System Calls — fork, exec, wait, zombie & orphan
3. Thread Synchronization — Producer–Consumer with semaphores & mutex
4. Dining Philosophers — Deadlock & Starvation
5. Deadlock Avoidance — Banker's Algorithm (with solved problems)
6. IPC using FIFO (named pipes) — Full duplex
7. IPC using Pipes — Full duplex Parent ↔ Child
8. Disk Scheduling — FCFS, SSTF, C-SCAN (with worked example)

How to use this: For each assignment, be able to (1) state the aim in one line, (2) explain the key system calls / algorithm, (3) write the core snippets from memory, and (4) answer the questions in the green box at the end.

Assignment 1 — Shell Programming

Aim: Learn shell programming.
Problem: Menu-driven script to handle a student database: a) Create b) View c) Insert d) Delete e) Modify f) Result of a student g) Exit.

Key theory

- **Shell** = command interpreter; the interface between user and kernel. It reads a command, finds the program (via `PATH`, typically `/bin`, `/usr/bin`), asks the kernel to run it, shows output, and prints the next prompt. Ends with `Ctrl+D` / `logout`.
- **Types of shells:** sh (Bourne), bash (Bourne Again — Linux default), csh (C-like syntax), tcsh (enhanced csh), ksh (Korn, superset of sh), rsh (restricted shell for guests). List known shells: `/etc/shells`.
- **Running a script:** `chmod +x sample.sh` then `./sample.sh` (or `bash sample.sh`).
- **test / [expr]** returns 0 for true, non-zero for false.
- **Comparison operators:** numbers: `-eq -ne -lt -le -gt -ge`; strings: `= != -z -n`; logical: `!` (NOT), `-a` (AND), `-o` (OR).
- **Positional parameters:** `$0` script name, `$1..$9` arguments, `$#` count, `$*` / `$@` all args, `$?` exit status of last command, `$$` PID of shell.

Constructs you must know

```
if [ $1 -gt 0 ]; then echo "positive"
elif [ $1 -lt 0 ]; then echo "negative"
else echo "zero"; fi

for i in 1 2 3 4 5; do echo "Welcome $i times"; done
for (( i=0; i<=5; i++ )); do echo $i; done

while true; do ... done          # infinite loop, exit with break
case $choice in
  1) pwd ;;
  2) date ;;
  *) echo "Invalid choice" ;;
esac
```

Core snippets for the student database

Store one record per line in a file, fields separated by a delimiter, e.g. `roll|name|marks`.

```
db="student.txt"
while true; do
  echo "1.Create 2.View 3.Insert 4.Delete 5.Modify 6.Result 7.Exit"
  read ch
  case $ch in
   1) read -p "DB name: " db; > $db ;;                 # create / empty file
   2) cat $db ;;                                         # view
   3) read -p "Roll Name Marks: " r n m
      echo "$r|$n|$m" >> $db ;;                          # insert (append)
   4) read -p "Roll: " r
      grep -v "^$r|" $db > tmp && mv tmp $db ;;         # delete
   5) read -p "Roll: " r; grep -v "^$r|" $db > tmp
      read -p "New Name Marks: " n m
      echo "$r|$n|$m" >> tmp; mv tmp $db ;;             # modify = delete + insert
   6) read -p "Roll: " r; grep "^$r|" $db ;;            # result
   7) exit ;;
   *) echo "Invalid choice" ;;
  esac
done
```

Remember: `>` overwrites, `>>` appends. `grep -v` prints lines that do *not* match — that's how delete works. `^` anchors the match to the line start so roll 1 doesn't match roll 11 (because of the `|` after it).

Answers — FAQs

1a. DOS vs Windows

> DOS is a single-user, single-tasking, command-line (CLI) 16-bit OS. Windows is a multitasking, GUI-based OS (32/64-bit) with multi-user support, networking and virtual memory.

1b. UNIX vs Linux

> UNIX is the original OS from AT&T Bell Labs (1969); most versions are proprietary (AIX, HP-UX, Solaris) and hardware-specific. Linux is a free, open-source UNIX-like kernel (Linus Torvalds, 1991) that runs on almost any hardware. Both share the same design ideas and commands.

1c. Windows vs Linux

> Windows: proprietary, paid, GUI-first, drive letters (C:\), backslash paths, case-insensitive file names. Linux: open-source, free, CLI and GUI, single directory tree from `/`, forward slashes, case-sensitive, strong permissions model, preferred for servers.

2. Five UNIX commands

> `ls` – list directory contents; `cd` – change directory; `pwd` – print working directory; `cp` / `mv` / `rm` – copy/move/remove files; `cat` – display/concatenate files; `grep` – search pattern in file; `chmod` – change permissions; `ps` – list processes; `who` – logged-in users.

3a. Wildcard patterns

> `*` matches any number of characters, `?` exactly one character, `[abc]` any one char in the set, `[a-z]` a range, `[!abc]` not in the set. E.g. `ls *.c`, `ls file?.txt`.

3b. Redirection

> Changes where input/output goes. `>` output to file (overwrite), `>>` append, `<` input from file, `2>` redirect errors (stderr). File descriptors: 0 = stdin, 1 = stdout, 2 = stderr.

3c. Pipes and filters

> A pipe `|` sends the output of one command as input to the next: `ls -l | wc -l`. A filter is a command that reads stdin, transforms it and writes stdout: `grep, sort, wc, head, tail, cut, tr, uniq`.

3d. File access permissions

> Three classes — user (owner), group, others — each with read (r=4), write (w=2), execute (x=1). Shown by `ls -l` as `-rwxr-xr--`. Changed with `chmod 754 file` or `chmod u+x file`; owner changed with `chown`.

4. What is the shell? (UNIX block diagram)

> UNIX is layered: **Hardware → Kernel → Shell → User / Applications**. The kernel manages processes, memory, files and devices. The shell sits between the user and the kernel; it interprets commands and asks the kernel to run them. Utilities/applications run on top.

5. Use of shell programming

> Automating repetitive tasks, system administration (backups, user creation, log monitoring), batch processing, combining many commands into one program, and writing install/startup scripts — without compiling.

Assignment 2 — Process Control System Calls

Aim: Demonstrate `fork`, `execve` and `wait` along with zombie and orphan states.
Part 1: Parent sorts integers with merge sort, child sorts with quick sort (after fork); show zombie and orphan states.
Part 2: Parent sorts an array and passes it to the child through `execve` command-line arguments; the new program does binary search on it.

Key theory

- **Process** = a program in execution. Each has a unique **PID** (`pid_t`, from `<sys/types.h>`). `getpid()` gives own PID, `getppid()` gives parent's PID.
- **Two ways to create a process:** `system("cmd")` (runs the command through `/bin/sh`; returns 127 if the shell can't run, −1 on other error) and `fork()`.
- **fork()** — `#include <unistd.h>`. Creates a child that is a copy of the parent's address space. Both continue from the statement after fork. **Returns 0 to the child, child's PID to the parent, −1 on failure.**
- **wait(&status)** — `<sys/wait.h>`. Parent blocks until a child ends; returns the child's PID (−1 on error). Macros: `WIFEXITED(status)` (did it exit normally?), `WEXITSTATUS(status)` (its exit code).
- **exec family** — replaces the current process image with a new program (same PID). Never returns on success; returns −1 on failure.

Function	Args as	Searches PATH?	Custom env?
execl	list	No	No
execlp	list	Yes	No
execle	list	No	Yes
execv	vector (array)	No	No
execvp	vector	Yes	No
execve	vector	No	Yes (the actual system call)

Memory trick: **l** = list, **v** = vector, **p** = PATH, **e** = environment.

Zombie vs Orphan

	Zombie	Orphan
What	Child has **terminated**, but parent hasn't called `wait()` yet; its entry stays in the process table (state **Z**, "defunct").	Child is **still running**, but its parent has terminated.
Fix	Parent calls `wait()`; or parent dies and init reaps it.	Automatically adopted ("re-parented") by **init / systemd (PID 1)**.
Demo	Child `exit(0)`; parent `sleep(60)`. Check with `ps -l`.	Parent exits; child `sleep(10)` then prints `getppid()` → 1.

Essential code bits

fork skeleton (Part 1)

```
pid_t pid = fork();
if (pid < 0)        { perror("fork"); exit(1); }
else if (pid == 0)  { quickSort(a, 0, n-1);  /* child */
                      printf("Child PID=%d PPID=%d\n", getpid(), getppid()); }
else                { mergeSort(a, 0, n-1);  /* parent */
                      wait(NULL);           /* prevents zombie */ }
```

Zombie & orphan demo

```
/* ZOMBIE: child dies first, parent doesn't wait */
if (fork() == 0) exit(0);
else { sleep(30); system("ps -l"); }   /* child shows state Z */

/* ORPHAN: parent dies first */
if (fork() == 0) { sleep(5); printf("New PPID = %d\n", getppid()); }
else exit(0);
```

Part 2 — passing sorted array via execve

```
/* parent: sort, then convert ints to strings for argv */
char *args[n+2];
args[0] = "./search";
for (i = 0; i < n; i++) {
    args[i+1] = malloc(10);
    sprintf(args[i+1], "%d", a[i]);
}
args[n+1] = NULL;                        /* argv must end with NULL */
if (fork() == 0) execve(args[0], args, NULL);
else wait(NULL);

/* search.c : int main(int argc, char *argv[]) */
for (i = 1; i < argc; i++) a[i-1] = atoi(argv[i]);
/* then standard binary search on a[0..argc-2] */
```

Merge sort & quick sort (the logic, in one line each)

- **Merge sort:** split array in half, recursively sort both halves, merge them. O(n log n) always, needs extra space, stable.
- **Quick sort:** choose a pivot, partition so smaller elements are left and larger are right, recurse on both sides. O(n log n) average, O(n^2) worst, in-place.
- **Binary search:** array must be sorted; compare with middle, discard half each step. O(log n).

Common viva trap: "How many processes does `fork(); fork(); fork();` create?" → 2^3 = **8 processes total** (7 new children). With n forks: 2^n.

Answers — likely viva questions

What does fork() return?

> 0 in the child, the child's PID (>0) in the parent, −1 on failure.

Difference between fork() and exec()?

> fork creates a *new process* (a copy of the caller). exec does not create a process; it *replaces* the code/data of the current process with a new program, keeping the same PID. They're usually used together: fork, then exec in the child.

Why use wait()?

> To make the parent wait for the child to finish, collect its exit status, and remove its entry from the process table (prevents zombies).

What happens to an orphan process?

> It is adopted by init/systemd (PID 1), which later calls wait() for it.

Can you kill a zombie with kill -9?

> No, it is already dead. You must make the parent call wait(), or kill the parent so init adopts and reaps the zombie.

fork vs vfork?

> vfork doesn't copy the parent's address space (the child borrows it) and guarantees the child runs first; the parent is suspended until the child calls exec or exit. It is faster when the child will exec immediately.

Why must argv end with NULL in execve?

> That's how the new program knows where the argument list ends (argc is calculated from it).

Does exec return?

> Only on failure (−1). On success the old program no longer exists, so there is nothing to return to.

What's in a PCB?

> PID, process state, program counter, CPU registers, scheduling info (priority), memory-management info, open files list, accounting info.

Process states?

> New → Ready → Running → Waiting/Blocked → Ready … → Terminated (and Zombie until reaped).

Assignment 3 — Producer–Consumer (Semaphores & Mutex)

Aim: Thread synchronization using counting semaphores and mutual exclusion using a mutex.
Objective: Implement the producer–consumer problem in C with counting semaphores and a mutex.

Key theory

- **Semaphore** = an integer used for signalling. Only 3 atomic operations: **initialize, decrement (wait / P / down), increment (signal / V / up)**. Decrement may block a process; increment may unblock one.
- **Binary semaphore:** value only 0 or 1 (like a lock). **Counting semaphore:** value 0…K, counts available resources. Has an integer count and a FCFS queue of blocked processes.
- Counting semaphore meaning (initial value 1): count = 1 → nobody in critical section; 0 → one process inside, none waiting; **negative −N** → one inside and N waiting. Waiting processes are **blocked, not busy-waiting**.
- **Mutex** = mutual exclusion lock; only the thread that locked it may unlock it. Used to protect the buffer (critical section).
- **Critical section:** code that accesses shared data; only one thread may run it at a time.
- **Race condition:** the result depends on the order in which threads run, because they access shared data without synchronization.

Producer–Consumer problem

Producers put items in a shared bounded buffer; consumers remove them. Rules: (1) only one thread touches the buffer at once, (2) producer must not add to a **full** buffer, (3) consumer must not remove from an **empty** buffer.

Variable	Type	Initial	Purpose
`empty`	counting semaphore	N (buffer size)	number of free slots
`full`	counting semaphore	0	number of filled slots
`mutex`	mutex	unlocked	mutual exclusion on buffer

Essential code bits

```
#include <pthread.h>
#include <semaphore.h>
#define N 5
int buffer[N], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    int item = rand() % 100;
    sem_wait(&empty);                  /* wait for a free slot   */
    pthread_mutex_lock(&mutex);        /* enter critical section */
    buffer[in] = item; in = (in + 1) % N;
    pthread_mutex_unlock(&mutex);
    sem_post(&full);                   /* one more filled slot   */
}

void *consumer(void *arg) {
    sem_wait(&full);                   /* wait for an item       */
    pthread_mutex_lock(&mutex);
    int item = buffer[out]; out = (out + 1) % N;
    pthread_mutex_unlock(&mutex);
    sem_post(&empty);                  /* one more free slot     */
}

/* main */
sem_init(&empty, 0, N);  sem_init(&full, 0, 0);
pthread_mutex_init(&mutex, NULL);
pthread_create(&p, NULL, producer, NULL);
pthread_create(&c, NULL, consumer, NULL);
pthread_join(p, NULL); pthread_join(c, NULL);
```

Compile: `gcc pc.c -o pc -pthread`

Order matters: Always do `sem_wait(&empty/full)` *before* locking the mutex. If you lock the mutex first and then block on `empty` (buffer full), the consumer can never get the mutex to free a slot → **deadlock**.

Answers — likely viva questions

Semaphore vs mutex?

> A mutex is a lock with ownership — only the locking thread can unlock it — and is used for mutual exclusion. A semaphore is a counter without ownership — any thread can signal it — and is used for signalling and counting resources. A binary semaphore is similar to a mutex but has no ownership.

What do sem_wait and sem_post do?

> `sem_wait` decrements the value; if it's 0 the thread blocks. `sem_post` increments it and wakes a waiting thread.

sem_init arguments?

> `sem_init(&sem, pshared, value)` — pshared = 0 means shared between threads of one process (non-zero = between processes); value = initial count.

Why are both semaphores and a mutex needed?

> Semaphores handle the full/empty conditions (counting); the mutex ensures only one thread modifies the buffer and the in/out indices at a time.

What is busy waiting?

> Repeatedly checking a condition in a loop while using the CPU. Semaphores avoid it by putting the process in a blocked queue.

Thread vs process?

> Threads of one process share code, data, heap and open files, but each has its own stack and registers. Threads are cheaper to create and switch between; processes are isolated from each other.

Requirements for a critical-section solution?

> Mutual exclusion, progress, and bounded waiting.

What is pthread_join?

> It makes the calling thread wait until the given thread finishes (similar to wait() for processes).

Assignment 4 — Dining Philosophers

Aim: Implement a deadlock solution to the Dining Philosophers problem to illustrate deadlock and/or starvation when many synchronized threads compete for limited resources.

Key theory

- Classical synchronization problem proposed by **Dijkstra**. 5 philosophers at a round table, 5 forks (one between each pair). Each one alternates **thinking** and **eating**; eating needs **both** left and right forks.
- Philosopher `i`: left fork = `i`, right fork = `(i+1) % N`.
- **Deadlock:** all 5 pick up their left fork at once → each waits for the right fork held by a neighbour → circular wait: P0→F1→P1→F2→…→P4→F0→P0. Nobody proceeds.
- **Starvation:** one philosopher keeps losing the race for forks while neighbours eat again and again.

Solution used: allow only N−1 philosophers at the table

A counting semaphore `room` initialized to **4** lets only 4 philosophers try to pick up forks. With 5 forks and 4 competitors, by the pigeonhole principle at least one gets both forks, eats, and releases them → **no circular wait**.

Algorithm per philosopher

Think → `sem_wait(room)` → lock left → lock right → eat → unlock right → unlock left → `sem_post(room)` → repeat.

Essential code bits

```
#define N 5
pthread_mutex_t forks[N];   /* one mutex per fork */
sem_t room;

void *philosopher(void *arg) {
    int id = *(int *)arg, left = id, right = (id + 1) % N;
    for (int i = 0; i < TIMES; i++) {
        printf("P%d thinking\n", id); sleep(1);
        sem_wait(&room);                    /* max N-1 inside */
        pthread_mutex_lock(&forks[left]);
        pthread_mutex_lock(&forks[right]);
        printf("P%d eating\n", id);  sleep(2);
        pthread_mutex_unlock(&forks[right]);
        pthread_mutex_unlock(&forks[left]);
        sem_post(&room);
    }
    return NULL;
}

/* main */
for (i = 0; i < N; i++) pthread_mutex_init(&forks[i], NULL);
sem_init(&room, 0, N - 1);
for (i = 0; i < N; i++) { id[i] = i;
    pthread_create(&ph[i], NULL, philosopher, &id[i]); }
for (i = 0; i < N; i++) pthread_join(ph[i], NULL);
/* pthread_mutex_destroy(...), sem_destroy(&room) */
```

Compile: `gcc dining_philosophers.c -o dp -pthread`. Output order varies because the OS schedules threads.

Other valid solutions (good to mention): (1) **Asymmetric** — odd philosophers pick left first, even pick right first; (2) **Resource ordering** — always pick the lower-numbered fork first; (3) pick up both forks only if both are free (inside a critical section / monitor solution).

Deadlock	Starvation
All involved threads wait forever	One/some threads wait very long
No one can proceed	Others continue executing
Caused by circular wait	Caused by unfair resource allocation
Each philosopher holds one fork and waits for another	One philosopher repeatedly fails to get both forks

Answers — likely viva questions

Why initialize the semaphore to N−1 (4)?

> With at most 4 philosophers competing for 5 forks, at least one can get both forks, so a circular wait can't form.

Which deadlock condition does this solution break?

> Circular wait.

Why is each fork a mutex?

> A fork can be used by only one philosopher at a time — mutual exclusion.

Does this solution prevent starvation?

> It prevents deadlock. Starvation is unlikely but not strictly guaranteed against, since it depends on how fairly the OS wakes up waiting threads.

Why pass &id[i] instead of &i to pthread_create?

> Because `i` changes in the loop; all threads would read the same changing address. Each thread needs its own stable value.

Why -pthread while compiling?

> It links the POSIX threads library (and sets the needed flags).

Assignment 5 — Banker's Algorithm (Deadlock Avoidance)

Aim: Implement the C program for the deadlock avoidance algorithm: Banker's Algorithm.
Objectives: deadlock & starvation, conditions for deadlock and its handling, the Banker's algorithm.

Key theory

Deadlock: a set of processes is deadlocked if each one is waiting for an event that only another process in the set can cause, so they all wait forever.

Four necessary conditions (Coffman, 1971) — all must hold

1. **Mutual exclusion** — a resource is held by only one process at a time.
2. **Hold and wait** — a process holds resources while requesting more.
3. **No preemption** — resources can't be forcibly taken; only released voluntarily.
4. **Circular wait** — a circular chain of processes, each waiting for a resource held by the next.

Data structures (n processes, m resource types)

Name	Size	Meaning
Available	m	free instances of each resource
Max	n × m	maximum demand of each process
Allocation	n × m	currently allocated
Need	n × m	**Need = Max − Allocation**

Safety algorithm

1. Work = Available; Finish[i] = false for all i.
2. Find an i with Finish[i] == false and Need[i] ≤ Work. If none, go to step 4.
3. Work = Work + Allocation[i]; Finish[i] = true; go to step 2.
4. If all Finish[i] are true → **safe state** (the order found is a safe sequence); otherwise **unsafe**.

Resource-request algorithm (process i requests Request[i])

1. If Request ≤ Need[i], continue; else **error** (exceeded maximum claim).
2. If Request ≤ Available, continue; else process must **wait**.
3. Pretend to allocate: Available −= Request; Allocation[i] += Request; Need[i] −= Request.
4. Run safety algorithm. Safe → **grant**. Unsafe → roll back, process **waits**.

Essential code bits (safety check)

```
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++) need[i][j] = max[i][j] - alloc[i][j];

for (j = 0; j < m; j++) work[j] = avail[j];
int count = 0;
while (count < n) {
    int found = 0;
    for (i = 0; i < n; i++) {
        if (!finish[i]) {
            for (j = 0; j < m; j++) if (need[i][j] > work[j]) break;
            if (j == m) {                          /* need <= work */
                for (j = 0; j < m; j++) work[j] += alloc[i][j];
                safeSeq[count++] = i; finish[i] = 1; found = 1;
            }
        }
    }
    if (!found) { printf("System is UNSAFE\n"); return; }
}
printf("SAFE. Sequence: "); /* print safeSeq[0..n-1] */
```

Solved Problem 1 — 5 processes, A=10, B=5, C=7

Process	Allocation (A B C)	Max	Need
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Available = (3, 3, 2)

(a) Can P4's request (3,3,0) be granted? (3,3,0) ≤ Need[4] (4,3,1) ✓ and ≤ Available (3,3,2) ✓. Pretend: Available = (0,0,2), Need[4] = (1,0,1). Now no process has Need ≤ (0,0,2) (P3 needs (0,1,1) — B is short). → **Unsafe. Not granted, P4 waits.**

(b) Can P0's request (0,2,0) be granted? (0,2,0) ≤ Need[0] ✓ and ≤ Available ✓. Pretend: Available = (3,1,2), Alloc[0] = (0,3,0), Need[0] = (7,2,3).
Work (3,1,2) → P3 [need 0,1,1] → (5,2,3) → P1 [1,2,2] → (7,2,3) → P0 [7,2,3] → (7,5,3) → P2 [6,0,0] → (10,5,5) → P4 [4,3,1] → (10,5,7). All finish.
→ **Safe sequence <P3, P1, P0, P2, P4>. Request GRANTED.**

> **Note — the lab manual has a mistake here:** it finds that every process can finish but then concludes "unsafe, P0 must wait". When all Finish[i] are true the state is **safe**, so the request **is granted**. If the examiner asks, explain using the rule in step 4 of the safety algorithm.

Solved Problem 2 — 5 processes, 4 resources

Process	Allocation	Max	Need
P0	0 0 1 2	0 0 1 2	0 0 0 0
P1	2 0 0 0	2 7 5 0	0 7 5 0
P2	0 0 3 4	6 6 5 6	6 6 2 2
P3	2 3 5 4	4 3 5 6	2 0 0 2
P4	0 3 3 2	0 6 5 2	0 3 2 0

Available = (2, 1, 0, 0)

(b) Safe? Work (2,1,0,0) → P0 → (2,1,1,2) → P3 → (4,4,6,6) → P4 → (4,7,9,8) → P1 → (6,7,9,8) → P2 → (6,7,12,12). **Safe: <P0, P3, P4, P1, P2>**

Request	Result	Reason
P0: (0,1,0,0)	**Error**	Request > Need[0] = (0,0,0,0); P0 already has its maximum.
P1: (0,1,0,0)	**Grant**	Available → (2,0,0,0); still safe: <P0, P3, P4, P1, P2>
P2: (0,1,0,0)	**Deny (wait)**	After P0, P3, P4 → Work (4,6,9,8); P1 needs B=7, P2 needs A=6 → unsafe.
P3: (0,0,0,1)	**Wait**	Request > Available (D = 0).

Solved Problem 3 — Deadlock detection

Available = (2,1,0,0). Allocation: P0 (0,0,1,0), P1 (2,0,0,1), P2 (0,1,2,0). Request: P0 (2,0,0,1), P1 (1,0,1,0), P2 (2,1,0,0).
Work (2,1,0,0) → P2 (request ≤ work) → (2,2,2,0) → P1 → (4,2,2,1) → P0 → (4,2,3,1). **All finish → no deadlock**, order <P2, P1, P0>.

Detection uses **Request** (current requests) instead of **Need** (future maximum) — that's the key difference from the safety algorithm.

Answers — FAQs

1. What is deadlock?

> A situation where a set of processes is blocked forever because each holds a resource and waits for a resource held by another process in the set.

2. Necessary and sufficient conditions for deadlock?

> Mutual exclusion, hold and wait, no preemption, circular wait. All four are *necessary*; together they are sufficient when there is a single instance of each resource type (with multiple instances, a cycle is necessary but not sufficient).

3. Deadlock avoidance vs prevention?

> **Prevention** makes deadlock impossible by design, by breaking at least one of the four conditions (e.g., request all resources at once to break hold-and-wait; number resources and request in order to break circular wait; allow preemption). **Avoidance** allows all four conditions but checks every request at run time and grants it only if the system stays in a **safe state** (Banker's algorithm). It needs to know each process's maximum need in advance. Other strategies: **detection & recovery** (let it happen, then kill/rollback processes) and **ignoring** it (ostrich algorithm, used by most desktop OSes).

Safe state vs unsafe state?

> Safe = there exists at least one order (safe sequence) in which all processes can finish. Unsafe does not mean deadlocked; it means deadlock is *possible*.

Why is it called "Banker's" algorithm?

> Like a bank that never lends its cash in a way that it can't satisfy all customers' maximum needs eventually.

Drawbacks of the Banker's algorithm?

> Needs max demand in advance, the number of processes and resources must be fixed, it is $O(m \times n^2)$ per request, and it is rarely used in real OSes.

Arguments for/against installing deadlock avoidance (manual's case study)

> **For:** deadlock would never occur, and all 5,000 jobs could still run despite the 20% longer turnaround. **Against:** deadlocks are rare (twice a month) and cheap (~10 jobs × $2 × half-done), so the 10% overhead on every job costs more than it saves.

What is a Resource Allocation Graph?

> A graph with process nodes (circles) and resource nodes (squares, dots = instances). Request edge P→R, assignment edge R→P. A cycle means deadlock if each resource has one instance; with multiple instances a cycle means deadlock is possible.

Assignment 6 — IPC using FIFO (Named Pipes)

Aim: Full-duplex communication between two **independent** processes using FIFOs. Process 1 accepts sentences and writes them on FIFO 1. Process 2 reads them, counts **characters, words and lines**, writes the result to a text file, and sends the file contents back on FIFO 2. Process 1 displays them.

Key theory

- **Unnamed pipe** (`pipe()`): exists only in memory, no name in the file system, works only between **related** processes (parent–child), half duplex.
- **FIFO / named pipe**: has a name (a special file) in the file system, so **unrelated** processes can use it. One-way data flow; first-in-first-out.
- Create: `mkfifo(const char *path, mode_t mode)` (e.g. mode 0666) — returns −1 on failure (e.g. it already exists). Also `mknod("fifo1", S_IFIFO|0666, 0)` or the shell command `mkfifo fifo1`.
- Use it like a file: `open`, `read`, `write`, `close`; delete with `unlink()`.
- **open() blocks** until the other end is opened too (reader waits for writer and vice versa) — a kind of handshake.
- Writes up to `PIPE_BUF` (4 KB on Linux) are **atomic**.
- **Full duplex = two FIFOs**, one for each direction.

Essential code bits

```
/* ---- process1.c (writer, then reader) ---- */
mkfifo("fifo1", 0666);  mkfifo("fifo2", 0666);
fd1 = open("fifo1", O_WRONLY);
printf("Enter sentences (end with #):\n");
/* read input into buf until '#' */
write(fd1, buf, strlen(buf) + 1);
close(fd1);

fd2 = open("fifo2", O_RDONLY);
read(fd2, result, sizeof(result));
printf("%s\n", result);
close(fd2);

/* ---- process2.c (reader, then writer) ---- */
fd1 = open("fifo1", O_RDONLY);
read(fd1, buf, sizeof(buf));
close(fd1);

int chars = 0, words = 0, lines = 0;
for (i = 0; buf[i] != '\0'; i++) {
    chars++;
    if (buf[i] == ' ' || buf[i] == '\n') words++;
    if (buf[i] == '\n') lines++;
}
fp = fopen("output.txt", "w");
fprintf(fp, "Chars=%d Words=%d Lines=%d\n", chars, words, lines);
fclose(fp);

/* read output.txt back into result[], then: */
fd2 = open("fifo2", O_WRONLY);
write(fd2, result, strlen(result) + 1);
close(fd2);
unlink("fifo1"); unlink("fifo2");
```

Run: compile both, then run them in **two separate terminals**: `./p1` and `./p2`.

Why two terminals? The processes are independent (not parent/child) — that's exactly why we need a *named* pipe instead of `pipe()`.

Answers — likely viva questions

Pipe vs FIFO?

> A pipe has no name, exists only while processes use it, and connects related processes. A FIFO has a name in the file system, persists until `unlink()`, and can connect unrelated processes. Both are one-way.

Why two FIFOs?

> A FIFO carries data in only one direction; full duplex needs one for each direction.

What happens if you open a FIFO for reading and no writer exists?

> `open()` blocks until some process opens it for writing (unless `O_NONBLOCK` is used).

What does ls -l show for a FIFO?

> The file type character **p**, e.g. `prw-r--r--`.

What is IPC? Name the mechanisms.

> Inter-Process Communication: ways for processes to exchange data. Pipes, FIFOs, message queues, shared memory, semaphores, signals, sockets.

Where is FIFO data stored?

> In a kernel buffer, not on disk. The file is only a name/entry point; its size shows as 0.

What does unlink() do?

> Removes the FIFO's name from the file system.

Assignment 7 — IPC using Pipes (Parent ↔ Child)

Aim: Full-duplex communication between parent and child using pipes. Parent writes a file's **pathname** on pipe 1; child reads it, opens the file, and writes the **file contents** on pipe 2; parent reads them and displays on standard output.

Key theory

- `int pipe(int fd[2);` creates a one-way channel: **fd[0] = read end**, **fd[1] = write end**. Returns 0 on success, −1 on failure.
- Pipes are created **before fork()**, so the child inherits both descriptors.
- Each process **closes the ends it doesn't use**. Otherwise `read()` never sees EOF (a writer still appears to exist) and the program can hang.
- Pipe capacity is limited (about 64 KB on Linux); a writer blocks when it is full.

Pipe	Descriptor	Used by
p1[1]	write end	Parent writes filename
p1[0]	read end	Child reads filename
p2[1]	write end	Child writes file contents
p2[0]	read end	Parent reads file contents

Essential code bits

```
int p1[2], p2[2];
pipe(p1); pipe(p2);                  /* before fork! */
pid = fork();

if (pid == 0) {                      /* ---- CHILD ---- */
    close(p1[1]); close(p2[0]);
    n = read(p1[0], filename, sizeof(filename) - 1);
    filename[n] = '\0';
    fd = open(filename, O_RDONLY);
    while ((n = read(fd, buffer, sizeof(buffer))) > 0)
        write(p2[1], buffer, n);
    close(fd); close(p1[0]); close(p2[1]);
    exit(0);
} else {                             /* ---- PARENT ---- */
    close(p1[0]); close(p2[1]);
    scanf("%99s", filename);
    write(p1[1], filename, strlen(filename));
    close(p1[1]);                    /* signals end of filename */
    while ((n = read(p2[0], buffer, sizeof(buffer))) > 0)
        write(STDOUT_FILENO, buffer, n);
    close(p2[0]);
    wait(NULL);
}
```

Compile/run: `gcc pipe_full_duplex.c -o pfd` → `./pfd` → enter `test.txt`.

Flow: Parent creates 2 pipes → fork → parent writes filename (p1) → child reads it, opens file, writes contents (p2) → parent reads and prints → wait() → exit.

Answers — Viva Questions

1. What is a pipe in an operating system?

> A one-way (half-duplex) IPC channel in kernel memory with a read end and a write end, used between related processes. Data comes out in the order it was written (FIFO).

2. Why are two pipes required for full-duplex communication?

> A single pipe carries data in only one direction. Parent→child needs one pipe and child→parent needs another; using one pipe both ways would let a process read back its own data.

3. What is the purpose of fork()?

> To create the child process. Because it's called after `pipe()`, the child inherits the pipe descriptors, which is what lets parent and child talk.

4. Difference between p1[0] and p1[1]?

> p1[0] is the read end (the child reads the filename from it); p1[1] is the write end (the parent writes the filename into it).

5. Why should unused pipe ends be closed?

> (a) `read()` returns EOF (0) only when *all* write ends are closed — if one stays open by mistake, the reader blocks forever. (b) It prevents accidental reads/writes in the wrong direction. (c) It frees file descriptors.

What is STDOUT_FILENO?

> The file descriptor for standard output, value 1 (stdin = 0, stderr = 2).

What does O_RDONLY mean?

> Open the file in read-only mode (flag for `open()`, from `<fcntl.h>`).

What happens when writing to a pipe whose read end is closed?

> The writer gets the **SIGPIPE** signal (and `write` fails with EPIPE).

Assignment 8 — Disk Scheduling

Title: Simulate disk scheduling algorithms: **FCFS, SSTF, C-SCAN**.
Objective: Understand how system efficiency increases by scheduling disk requests.
Input: number of cylinders, sequence of requests, (initial head position). **Output:** order of servicing, total head movement / average seek time.

Key theory

- **Seek time:** time to move the disk arm/head to the required track (cylinder). The biggest part of disk access time, so algorithms try to minimize total head movement.
- **Rotational latency:** time for the required sector to rotate under the head. **Transfer time:** time to actually read/write data.
- **Access time** = seek time + rotational latency + transfer time.
- Total head movement = Σ |next − current|. Average seek = total / number of requests.

Algorithm	How it works	Pros	Cons
FCFS	Serve requests in arrival order	Simple, fair, no starvation	Large head movement
SSTF	Always serve the request nearest to the current head	Much less movement than FCFS	Starvation of far requests; not optimal
SCAN (elevator)	Move in one direction to the disk end serving requests, then reverse	No starvation, good throughput	Ends of disk wait longer
C-SCAN	Move one direction to the end, jump back to cylinder 0 without serving, continue in the same direction	**Uniform waiting time**	Extra movement for the return jump
LOOK / C-LOOK	Like SCAN / C-SCAN but turn at the last request instead of the disk end	Less movement than SCAN/C-SCAN	Slightly more logic

Worked example (standard textbook)

Cylinders 0–199, head at **53**, queue: **98, 183, 37, 122, 14, 124, 65, 67**

Algorithm	Service order	Total movement	Avg (÷8)
FCFS	53→98→183→37→122→14→124→65→67	**640**	80
SSTF	53→65→67→37→14→98→122→124→183	**236**	29.5
SCAN (toward 199)	53→65→67→98→122→124→183→199→37→14	**331**	41.4
C-SCAN (toward 199)	53→…→183→199→0→14→37	**382** (incl. 199-cylinder return)	47.75

C-SCAN: (199−53) + (199−0) + 37 = 146 + 199 + 37 = 382. Some examiners don't count the return jump → 146 + 37 = 183. State which convention you use.

Note: The manual's SSTF steps describe "serve all lower tracks in decreasing order, then higher tracks in increasing order" — that is actually SCAN/LOOK behaviour. True SSTF picks the **closest pending request each time**. Implement the true version, as below.

Essential code bits

```
/* FCFS */
for (i = 0; i < n; i++) { total += abs(req[i] - head); head = req[i]; }

/* SSTF */
for (k = 0; k < n; k++) {
    int min = INT_MAX, idx = -1;
    for (i = 0; i < n; i++)
        if (!done[i] && abs(req[i] - head) < min) { min = abs(req[i] - head); idx = i; }
    total += min; head = req[idx]; done[idx] = 1;
}

/* C-SCAN (moving toward higher cylinders) */
sort(req, n);
for (i = 0; i < n; i++) if (req[i] >= head) { total += req[i] - head; head = req[i]; }
total += (disk_size - 1) - head;      /* go to end         */
total += (disk_size - 1);             /* jump back to 0    */
head = 0;
for (i = 0; i < n; i++) if (req[i] < start) { total += req[i] - head; head = req[i]; }

printf("Average seek = %.2f\n", (float)total / n);
```

Answers — FAQs

1. What do you mean by seek time?

> The time taken by the read/write head to move from its current track to the track that holds the requested data. Disk scheduling aims to minimize the total seek time.

2. Difference between SCAN and C-SCAN?

> SCAN serves requests in **both** directions: it goes to one end, reverses, and serves on the way back (like an elevator). C-SCAN serves in **only one** direction: on reaching the end it jumps straight back to the start without serving any requests, then continues the same way. C-SCAN gives more **uniform waiting times**, because SCAN's middle cylinders get visited twice as often as the edges.

3. Comparison of the four algorithms

> **FCFS:** simplest and fair, but highest head movement (640 in the example). **SSTF:** lowest movement here (236) but can starve distant requests. **SCAN:** no starvation, good for heavy load (331). **C-SCAN:** slightly more movement (382) but the most uniform response time. Under heavy load SCAN/C-SCAN (or LOOK/C-LOOK) are preferred; under light load SSTF is good; FCFS is fine only when there are few requests.

Which algorithm suffers from starvation?

> SSTF.

Why is SCAN called the elevator algorithm?

> The head moves like a lift — all the way in one direction serving floors, then reverses.

Application

> Multimedia and database systems, where reducing disk access time improves throughput.

Quick-revision cheat sheet

Call / term	Remember
`fork()`	0 → child, PID → parent, −1 → error
`exec*()`	Replaces process image, same PID; returns only on error
`wait(&st)`	Blocks until child ends; reaps zombie
`pipe(fd)`	fd[0] read, fd[1] write; related processes only
`mkfifo(path, 0666)`	Named pipe; unrelated processes; delete with unlink
`sem_init(&s,0,v)`	0 = between threads; v = initial value
`sem_wait / sem_post`	P (decrement, may block) / V (increment, may wake)
`pthread_create / join`	Start a thread / wait for it to finish
Need	Max − Allocation
Deadlock conditions	Mutual exclusion, Hold & wait, No preemption, Circular wait
Producer–consumer semaphores	empty = N, full = 0, mutex = 1
Dining philosophers fix	Semaphore room = N−1 → breaks circular wait
Compile threads	`gcc file.c -o out -pthread`